POLITÉCNICO GRANCOLOMBIANO
INSTITUCIÓN UNIVERSITARIA

FACULTAD DE INGENIERÍA, DISEÑO E INNOVACIÓN

Unidad 4 / Escenario 7

Lectura fundamental

Metodologías y procesos de arquitectura de *software*

Contenido

1 Revisión del plan de desarrollo

2 Procesos, metodologías, marcos conceptuales

Palabras clave: QAW, FRUPS+, ACDM, VIEWPOINT, QAWM

1. Revisión del plan de desarrollo

La calidad de la arquitectura de software se puede evaluar principalmente mediante 3 aspectos: 1) la pertinencia, 2) la manera en que se llega a la solución y 3) los artefactos o el producto final. (ISO/IEC/IEEE42010, 2011).

En esta etapa del curso es importante revisar tanto el proceso como el producto de arquitectura. Por lo general, se usan indistintamente las palabras procesos, metodologías, métodos y marcos conceptuales para referirse tanto a herramientas como artefactos (productos). A través de esta unidad se realizará precisión sobre los dos panoramas.

Para esto, se presentan los modelos de proceso más utilizados en la arquitectura de software a través del escenario 7 y, luego, en el escenario 8 se explican diversas formas sistemáticas usadas para construir o elaborar los distintos artefactos y las formas de hacerlo. En cada una de las fases o partes de los procesos, marcos conceptuales, metodologías o frameworks de trabajo presentados, existen vistas o conjuntos de artefactos que resuelven converses específicos o correspondientes a las necesidades de dichas fases, que es preciso definir.

2. Procesos, metodologías, marcos conceptuales

El proceso: es una serie de actividades, tareas o pasos consecutivos o encadenados a través de los cuales se realiza la transformación material y conceptual de datos o de información. Es el cambio en la entrada que produce una determinada salida.

El método: secuencia de acciones que comprenden diversas técnicas y procesos.

La metodología: incluye al método, pero, además, define roles, fases, iteraciones, recomendaciones y artefactos.

Los marcos conceptuales: son el respaldo teórico de la metodología; proveen un conjunto de conceptos que deben considerarse al documentar la arquitectura o aplicar las técnicas.

2.1. El Método de Diseño Centrado en Arquitectura (ACDM)

El ACDM es una metodología desarrollada por Anthony Lattanze (2010), de la Universidad de Carnegie Mellon, que propone facilitar el proceso de diseño de una solución e integrarla a los procesos de desarrollo comúnmente usados; expone técnicas y estructuras, y define roles para diseñar y usar la arquitectura para la construcción de artefactos.

Se construye bajo un proceso iterativo y de mejora continua que permite que el equipo esté seguro de poder implementar el producto.

El diseño arquitectural es usado para planear, estimar y seguir las actividades de construcción y allí radican sus principales fortalezas: la capacidad de identificar, organizar y analizar, de manera temprana, los concerns y drivers de la arquitectura en las primeras etapas del desarrollo de software.

2.1.1. Estados del método ACDM

En las primeras etapas, 1 y 2, se establecen los requerimientos (drivers motivadores o concerns) de la arquitectura y se valida su alcance; en la etapa 3 se crea o se refina la arquitectura, lo que constituye el diseño inicial; luego, en la etapa 4, se revisa la arquitectura; en el siguiente paso (etapa 5) el equipo decide si la arquitectura debe o no ser refinada, es decir, "si va o no va", y, en el caso de que sí, en el paso 6 se realizan experimentos para solucionar dudas del equipo. Una vez se termina el experimento, se retorna al paso 3; si la arquitectura no requiere ser refinada, se planea la producción y se ejecuta. Los pasos 1 a 6 se realizan sin la certeza del tiempo a invertir, mientras que en las dos últimas fases existe un tiempo establecido o cuantificable.

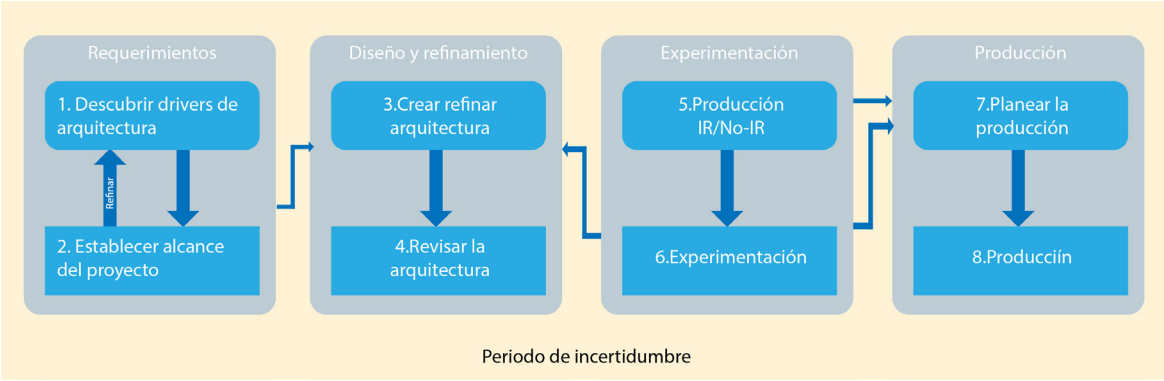

Figura 1. Estados e iteraciones del método de diseño centrado en la arquitectura.

Fuente: Elaboración propia (2018). Modificado de Lattanze (2010)

2.1.2. Roles del método ACDM (Craig, 1999)

Ingeniero gerente: aunque el volumen de trabajo sea mayor en las primeras y últimas entapas, es el encargado y responsable de coordinar todo el diseño y el desarrollo del sistema. Crea el plan maestro de diseño inicial y ayuda al ingeniero de requisitos en la planificación de los controles de arquitectura. De manera constante, se encarga de la actualización y replanificación del plan maestro de diseño basado en datos reales de los talleres de creación; "cuando el proyecto sale bien es culpa de quienes lo implementan, cuando el proyecto sale mal es culpa del gerente".

Ingeniero de soporte: es el responsable de poner a punto todo y mantener las herramientas necesarias para la elaboración del proyecto; debe trabajar en conjunto con el ingeniero de requerimientos para obtener las demandas de todos los stakeholders, esto es instalación, configuración y mantenimiento de las herramientas necesarias y de respaldo para el diseño y elaboración de la arquitectura y del resto del proyecto.

Arquitecto jefe: su función principal es el diseño y mantenimiento de todo el sistema, el cual se materializa a través de un documento. Par tal fin, trabaja con el ingeniero de requisitos para obtener los requisitos de las partes interesadas, en especial, los no funcionales. El arquitecto jefe es responsable de la captura de los drivers de arquitectura, de coordinar el diseño del sistema y de diseñar, revisar, refinar y documentar la arquitectura, preferiblemente, en todo el ciclo de vida del *software*.

Científico jefe: asiste al arquitecto jefe con detalles técnicos que puedan impactar el sistema y el clear box test, que, esencialmente, es un análisis directo del código fuente. Debe trabajar junto con el ingeniero de requisitos para obtener los requisitos de los *stakeholders*, sin descuidar los drivers arquitectónicos. Debe capturar y documentar la colección de drivers de arquitectura en bruto.

Ingeniero de requisitos: lidera la documentación de los requerimientos y planifica, coordina y facilita la obtención de drivers arquitectónicos no solo durante la primera etapa, sino en la evolución de estos. Coordina las reuniones de consolidación de drivers arquitectónicos y compila el documento de requerimientos del sistema.

Ingeniero de procesos de calidad: ayuda a los equipos de ingeniería de requerimientos a capturar y documentar la colección de drivers de arquitectura en bruto. Es el dueño del modelo o metodología ACDM (ver tabla 1) y de las adiciones que se le puedan hacer a esta. Trabaja con el ingeniero de requisitos para obtener los atributos de calidad a partir de los requisitos y drivers de arquitectura. Coordina una revisión del documento de drivers de arquitectura con el fin de asegurar la alineación estratégica, para lo cual está en contacto constante con los *stakeholders*.

Ingeniero de producción: no se deben confundir con otros ingenieros (ingenieros de *software*, ingenieros electrónicos, ingeniero de TI, mecánico o químico); tiene su función en todo el proceso, pero específicamente en la etapa 6 (ver figura 1). Debe ser una persona muy experimentadas en la implementación para poder apoyar procesos de prueba y error que se dan en esta etapa y, así, evitar sesgos, dada la dificultad que conlleva la implementación de una herramienta. Esa experiencia es de gran utilidad, razón por la cual este ingeniero debe ayudar e involucrarse con los ingenieros de requisitos y arquitectos en la captura, reformulación y obtención de drivers arquitectónicos.

2.1.3. Documentación en el método ACDM

Si bien ACDM especifica las etapas para la elaboración de la documentación de la arquitectura, entregables y roles específicos, no hace énfasis en el uso de una notación particular; empero, sugiere organizar la documentación de acuerdo con el esquema presentado en la tabla 1.

Tabla 1. Estructura del documento de arquitectura sugerida para ACDM. Modificado de Lattanze (2010)

1. Portada
2. Descripción del documento
3. Descripción de proyecto
4. *Drivers* arquitectónicos
5. Contexto del sistema
6. Nivel n de descomposición del sistema
6.1 Descripción
6.2 Representación primaria
6.3 Razones de la descomposición
6.4 Catálogos de elementos y relaciones
6.5 Interfaces
7. Mapeo entre perspectivas
8.Índices por palabras, glosarios y acrónimos

Fuente: Elaboración propia (20118)

2.2. Quality Attribute Workshops Method (QAWM)

Más que un método para el desarrollo de la arquitectura del sistema, QAWM es un taller de un día que contribuye a la primera fase de un proceso de arquitectura, que consiste en la captura de requerimientos, escenarios y drivers de arquitectura. Adicionalmente, este método incluye una reunión para el aseguramiento de la calidad. El objetivo de QAWM es tratar de determinar, lo más pronto posible y de forma ordenada, los atributos de calidad del sistema que servirán como base fundamental (drivers de arquitectura del sistema); es importante definir claramente los requerimientos antes de que la arquitectura sea desarrollada.

2.2.1. Estados del método QAWM

QAWM es un método que facilita el descubrimiento de los atributos de calidad para sistemas y software intensivos, para los cuales es imprescindible la definición clara de los atributos de calidad antes de la especificación de la arquitectura. El objetivo es que el conjunto de stakeholders identifiquen y prioricen los atributos y escenarios desde todo punto de vista, establezcan los críticos y los articulen de manera temprana al diseño de la arquitectura.

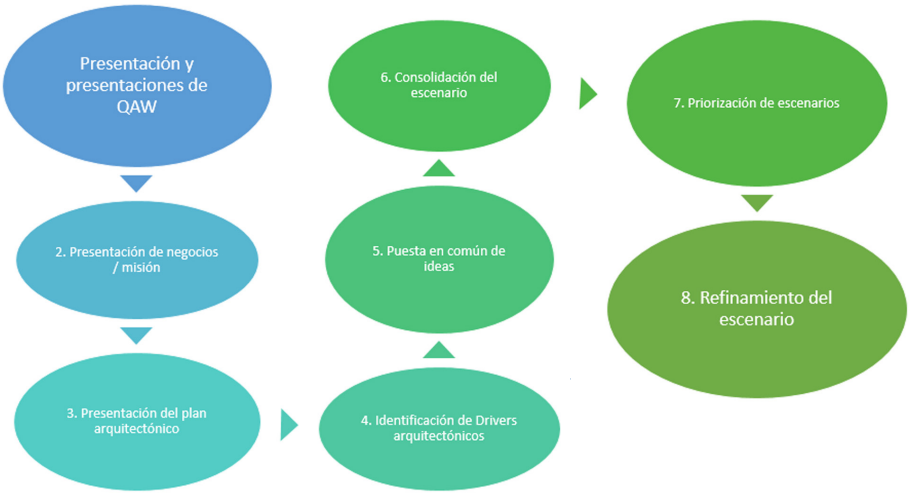

Figura 2. Modelos de proceso del workshop de atributos de calidad (QAWM). Modificado de Barbacci et al. (2003)
Fuente: Elaboración propia (2018).

2.2.2. Roles del método QAWM

No se definen funciones especiales para el taller, dado que es un evento conjunto de un equipo en el que pueden estar involucrados de 5 a 7 personas. Se proponen 2 facilitadores para la captura de la información, con los artefactos presentados en la siguiente sección (ver tablas 2 y 4). Luego se debe organizar la información, tarea a cargo de los siguientes 2 roles definidos en la metodología, que se explican a continuación.

Lead QAW: tiene claridad sobre el método y garantiza que tanto los pasos como los artefactos sean diligenciados y llevados a cabo correctamente.

Escritor de QAW: es quien diligencia todos los formatos. Su responsabilidad es consolidar el documento hasta el paso 7; luego, bajo supervisión del Lead QAW, asegura la finalización y generación de todas las fichas y el documento.

2.2.3. Documentación en el método QAWM

El encabezado del documento son el título del taller y la fecha; quizás sea necesario un identificador de acuerdo con las políticas de gestión documental. En la primera sección es importante tener registro de los asistentes y tomar notas según corresponda en la segunda y tercera sección (ver tabla 3); debe realizarse un listado de los atributos de calidad a los que se le dará prioridad o se someterán a consideración, incluyendo los problemas asociados y las distintas anotaciones que se deriven del taller.

En la sección 4, "Identificación de drivers arquitectónicos", se consignan las aclaraciones y correcciones de la lista de los drivers arquitectónicos. Para obtenerlos se deben presentar uno a uno, no necesariamente en orden los drivers consignados en los ítems 2 y 3. Esa lista ayudará a los facilitadores a garantizar la cobertura total durante la tormenta de ideas del escenario.

Tabla 2. Recolección de escenarios. Modificado de Barbacci et al. (2003)

ID Escenario	Descripción del escenario	Votos
#1		
#2		
#3		
#4		
#5		
……		
Total:		

Fuente: Elaboración propia (20118)

Desde la sección 5 a la 7 se utiliza la misma estructura (ver tabla 2), insertando filas según sea necesario. En la sección 5 se redactan los escenarios crudos propuestos por cualquiera de los stakeholders, usando el formato e incluyendo una valoración parcial y total; luego, (sección 6) se deben fusionar escenarios similares y duplicados, y presentarlos. Finalmente (sección 7), cada parte interesada debe, por lo menos, realizar una votación equivalente al 30 % del número total de escenarios.

Tabla 3. Estructura del documento final del taller QAW.

- Título
- Historial de cambios
- Tabla de contenido
- Índice de figuras

1. Alcance
2. Referencias
3. Arquitectura de *software*
4. Objetivos de la arquitectura y restricciones
5. Arquitectura lógica
6. Arquitectura de procesos
7. Arquitectura de desarrollo
8. Arquitectura física

Fuente: Elaboración propia (2018). Modificado de Barbacci et al. (2003)

Para la última sección, "Refinamiento del escenario", quizás la más extensa (ver tabla 4), se presenta el inventario total de escenarios, incluyendo múltiples detalles, tales como: ¿Cuánto tiempo?, ¿Con qué frecuencia?, ¿Cuándo?, ¿Quién?, y así sucesivamente.

Tabla 4. Ejemplo de tabla de refinamiento de escenario.

Refinamiento del escenario para el escenario N		
Escenario (s):		Cuando un sistema de apertura de puerta de garaje detecta un objeto en el camino de la puerta, detiene la puerta en menos de un milisegundo.
Objetivos comerciales:		sistema más seguro; producto rico en características
Atributos de calidad relevantes:		Rendimiento Seguridad
Componentes del escenario	**Estímulo:**	Un objeto está en el camino de una puerta de garaje.
	Fuente del Estímulo:	objeto externo al sistema, como una bicicleta
	Medio ambiente:	La puerta del garaje está en proceso de cierre.
	Artefacto (si se conoce):	sensor de movimiento del sistema, componente de software de control de movimiento
	Respuesta:	La puerta del garaje deja de moverse.
	Medida de la Respuesta:	un milisegundo
Inventario de Preguntas:		¿Qué tamaño debe tener un objeto antes de que sea detectado por el sistema de censado?
Problemas:		Puede ser necesario capacitar a los instaladores para evitar el mal funcionamiento y evitar posibles problemas legales.

Fuente: Elaboración propia (2018). Modificado de Barbacci et al. (2003)

2.3. El modelo de vista "4 + 1" de la arquitectura de software

Más que una metodología, el modelo 4+1, de Philippe Kruchten, es una clasificación o modelo de organización de las distintas vistas UML para la completa descripción del software y su arquitectura. Esto permite resolver separadamente cada uno de las preocupaciones o drivers de los distintos stakeholders de la arquitectura de *software*, entre ellos el usuario final, gerentes de proyecto, programadores, ingenieros de sistemas etc., Propone una aproximación iterativa para lograr consolidar las 5 vistas presentadas en el siguiente diagrama:

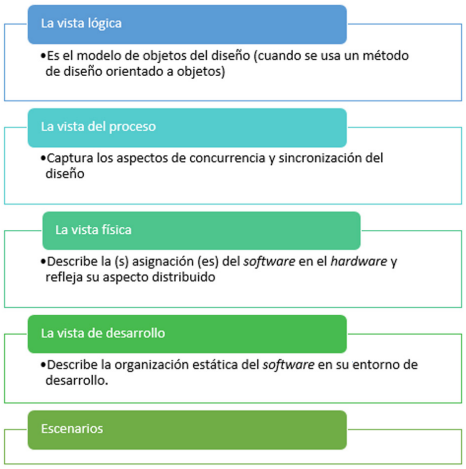

Figura 5. Modelo de vistas 4+1 de arquitectura de software.

Fuente: Elaboración propia (2018). Modificado de Kruchten (1995)

Tabla 5. Estructura "4+1", Documento de arquitectura sugerida.

• Título
• Historial de cambios
• Tabla de contenido
• Índice de figuras
1. Alcance
2. Referencias
3. Arquitectura de *software*
4. Objetivos de la arquitectura y restricciones
5. Arquitectura lógica
6. Arquitectura de procesos
7. Arquitectura de desarrollo
8. Arquitectura física
9. Escenarios
10. Tamaño y desempeño
11. Calidad
Apéndices
A. Acrónimos y abreviaciones
B. Definiciones
C. Principios de diseño

Fuente: Elaboración propia (2018). Modificado de Kruchten (1995)

¿Sabía qué...?

Philippe Kruchten, profesor de Ingeniería de software de University of British Columbia, Canadá, ha escrito cerca de 249 artículos entre 1977 y 2004, hecho que lo convierte en un referente en su área. Varios de los artículos se encuentran en las columnas de IEEE software Magazine. Puedes conocer más acerca de este autor en: https://philippe.kruchten.com/

2.4. Functionality, Usability, Reliability, Performance, Supportability y (FURPS+)

El modelo FURPS+ contempla tanto requerimientos funcionales (1) como requerimientos no funcionales (2-5) y 5 restricciones adicionales (+), para un total de nueve características como factores de calidad, que son los que le dan su nombre.

Tabla 6. Características generales de FURPS.

ID	Sigla	Tipo	Nombre	Descripción
1	F	(RF)	Funcionalidad	Características de sistemas Capacidades, algunos aspectos de seguridad.
2	R	(NRF)	Usabilidad	Factores humanos (interacción), ayuda, estética, consistencia, documentación.
3	U	(NRF)	Confiabilidad	Capacidad y frecuencia de fallo Recuperabilidad, precisión, grado de predicción.
4	P	(NRF)	Desempeño	Velocidad, eficiencia. Consumo (uso de recursos) Productividad, tiempo de respuesta.
5	S	(NRF)	Soporte	Adaptabilidad, extensibilidad, mantenibilidad, compatibilidad, configurabilidad Internacionalización.
6	(+)	Restricción	Empaquetamiento	Forma de distribución.
7	(+)	Restricción	Implementación	Limitación de recursos, lenguaje, herramienta.
8	(+)	Restricción	Interface-Físicas	*Hardware*, restricciones de interacción con sistemas externos, no es GUI.
9	(+)	Restricción	Operación	Pautas administrativas, puesta en marcha, gestión del sistema.
10	(+)	Restricción	Legales	Licencia, derechos de autor.

Fuente: Elaboración propia (2018). Modificado de Craig (1999)

Cómo mejorar...

En la sección final de este documento, se listan otra serie de metodologías, modelos de proceso y frameworks o marcos conceptuales que son importantes investigar para tomar lo mejor de cada mundo para las propuestas de su proyecto de investigación formativo (Vistas y más allá, puntos de vista y perspectivas, el estándar ISO/IEC/IEEE 42010:2011, el modelo 4 Vistas de Siemens, Rational ADS, entre otros).

2.5. Listado de metodologías, modelos de proceso y frameworks:

- Vistas y más allá
- Puntos de vista y perspectivas
- El estándar ISO/IEC/IEEE 42010:2011
- El modelo 4 Vistas de Siemens
- Rational ADS

2.6. Comparación de los métodos y marcos conceptuales

Como anexo, se presenta la siguiente tabla que compara metodologías, frameworks y procesos de arquitectura, y en la que se exponen algunos de los métodos, modelos y marcos conceptuales abordados en esta lectura fundamental.

Tabla 7. Comparación de metodologías, frameworks y procesos de arquitectura

	Vistas y más allá	4+1 Vistas	Puntos de vista y perspectivas	Método de diseño centrado en la arquitectura (ACDM)
Tipo	Método	Modelo	Marco Conceptual	Drivers de Arquitectura
Mecánica y enfoque	Definido como una secuencia de pasos que soporta la identificación de las vistas a documentar	Puede ser utilizado dentro del contexto de un método de desarrollo para soportar la especificación y documentación de vistas arquitectónicas	Modelo que puede ser utilizado en el contexto de un método de desarrollo para soportar la especificación y documentación de vistas arquitectónicas	Método iterativo e incremental que soporta la identificación y documentación de vistas arquitectónicas
Provee un proceso que especifica las actividades, roles y artefactos de entrada y salida	Sí	No aplica	No aplica	Sí
Participantes	Arquitecto de software	No aplica	No aplica	Se asume la participación de los siguientes roles: administrador del proyecto, arquitecto de software líder, líder científico, ingeniero de requerimientos, ingeniero de calidad, ingeniero de soporte e ingeniero de producción.
Entradas	Conjunto de interesados en el sistema e información proveniente de la etapa de diseño de la arquitectura	No aplica	No aplica	Drivers de arquitectura
Salidas	Lista de vistas a documentar junto con el orden en que deben ser elaboradas	No aplica	No aplica	Conjunto de vistas arquitectónicas documentadas
Criterios de terminación	Tener un conjunto de vistas a documentar y el orden en que deben ser elaboradas	No aplica	No aplica	Todas las vistas arquitectónicas están documentadas

Considera las tres vistas fundamentales: lógicas, de comportamiento y físicas	Método	Modelo	Marco Conceptual	Drivers de Arquitectura
Considera vistas adicionales	Definido como una secuencia de pasos que soporta la identificación de las vistas a documentar	Puede ser utilizado dentro del contexto de un método de desarrollo para soportar la especificación y documentación de vistas arquitectónicas	Modelo que puede ser utilizado en el contexto de un método de desarrollo para soportar la especificación y documentación de vistas arquitectónicas	Método iterativo e incremental que soporta la identificación y documentación de vistas arquitectónicas
Considera información	Sí	No aplica	No aplica	Sí
adicional del sistema que complementa la de las vistas (por ejemplo, descripción del sistema, relación entre vistas, glosarios, restricciones de negocio y drivers arquitectónicos)	Arquitecto de software	No aplica	No aplica	Se asume la participación de los siguientes roles: administrador del proyecto, arquitecto de software líder, líder científico, ingeniero de requerimientos, ingeniero de calidad, ingeniero de soporte e ingeniero de producción.
Provee soporte para documentar atributos no funcionales específicos	No	No	Sí	No
Da soporte para identificar o priorizar las vistas a documentar	Sí	No	No	Sí
Sugiere cómo organizar la documentación generada	Sí	Sí	No	Sí
Notación utilizada	No indica	No aplica	Hace determinado énfasis en el uso en UML	No indica

Fuente: Elaboración propia (2018). Modificado de Cervantes Maceda, Velasco Elizondo y Castro Careaga (2016).

En síntesis...

Existen una gran variedad de metodologías, marcos de referencia y modelos de proceso que sirven de apoyo para el arquitecto de software. Muchos de ellos se complementan con otros que son redundantes, pero con seguridad cada uno tiene una característica diferenciadora que lo hace especial y que contribuye a mejorar la comunicación y organización de las actividades, artefactos y roles involucrados en la arquitectura de software. En esta lectura se abordaron varios temas: la metodología y procesos (ACDM), las fases iniciales y el aseguramiento de la calidad (QAWM y FRUPS+), las vistas y su organización (4+1) y, finalmente, una serie de aspectos considerados de manera transversal.

Referencias bibliográficas

Barbacci, M. R., Ellison, R., Lattanze, A. J., Stafford, J. A., Weinstock, C. B. & Wood, W. G. (2003). Quality Attribute Workshops (QAWs). Pittsburgh, PA: Software Engineering Institute.

Cervantes Maceda, H., Velasco Elizondo, P. y Castro Careaga, L. (2016). Arquitectura de software. Conceptos y ciclo de desarrollo. México: Cengage Learnig.

Craig, L. (1999). UML y patrones: introducción al análisis y diseño orientado a objetos. PEARSON.

ISO/IEC/IEEE 42010. (2011). Systems and software engineering — Architecture description. IEEE Xplore: INTERNATIONAL STANDARD.

Kruchten, P. (1995). The "4+1" View Model of Architecture. IEEE Software, 12(6), 42-50.

Lattanze, A. J. (2010). Architecting Software Intensive Systems: A Practitioner's Guide. Boca Raton, FL, EUA: CRC Press.

Referencias de figuras

Barbacci, M. R., Ellison, R., Lattanze, A. J., Stafford, J. A., Weinstock, C. B. & Wood, W. G. (2003). Quality Attribute Workshops (QAWs). Pittsburgh, PA: Software Engineering Institute.

Kruchten, P. (1995). The "4+1" View Model of Architecture. IEEE Software, 12(6), 42-50.

Lattanze, A. J. (2010). Architecting Software Intensive Systems: A Practitioner's Guide. Boca Raton, FL, EUA: CRC Press.

Referencias de tablas

Barbacci, M. R., Ellison, R., Lattanze, A. J., Stafford, J. A., Weinstock, C. B. & Wood, W. G. (2003). Quality Attribute Workshops (QAWs). Pittsburgh, PA: Software Engineering Institute.

Cervantes Maceda, H., Velasco Elizondo, P. y Castro Careaga, L. (2016). Arquitectura de software. Conceptos y ciclo de desarrollo. México: Cengage Learnig.

Craig, L. (1999). UML y patrones: introducción al análisis y diseño orientado a objetos. PEARSON.

Kruchten, P. (1995). The "4+1" View Model of Architecture. IEEE Software, 12(6), 42-50.

Lattanze, A. J. (2010). Architecting Software Intensive Systems: A Practitioner's Guide. Boca Raton, FL, EUA: CRC Press.

INFORMACIÓN TÉCNICA

FACULTAD DE
INGENIERÍA, DISEÑO
E INNOVACIÓN

Módulo: Arquitectura del *software*

Unidad 4: Procesos y documentación de la arquitectura

Escenario 7: Metodologías y procesos de Arquitectura de *software*

Autor: Diego Iván Oliveros Acosta

Asesor Pedagógico: Judy Fernanda Villanueva

Diseñador Gráfico: Diego Andres Calderón

Asistente: Ginna Quiroga

Este material pertenece al Politécnico Grancolombiano. Por ende, es de uso exclusivo de las Instituciones adscritas a la Red Ilumno. Prohibida su reproducción total o parcial.